

Storing Images and BLOB files in SQL Server Part 3

By Don Schlichting

Storing Images and BLOB files in SQL Server Part 3

Introduction

In the first two articles of this series, BLOBs were defined along with scenarios of when to store them inside SQL Server rather than on the file system. Several different types of VARBINARIES were introduced, including the VARBINARY(MAX) data type which will be the focus of this article. The first code example loaded an image into the database then retrieved it using an ASPX page with a Binary Write. In this article, a web-based application will be created for inserting images into SQL Server. In addition, the examples from the first two articles will be expanded and improved.

SQL Test Database

To begin, create a test database and table for storing images using the following TSQL script:

```
USE master;
GO
CREATE DATABASE BLOBTest3;
GO

USE BLOBTest3;
GO

CREATE TABLE BLOBFromWeb
(
  BLOBData varbinary(max)
);

GO
```

The script created a single database, containing a single table, containing a single column. The test column "BLOBData" uses the MAX keyword allowing it to store binaries of any size.

Next create a stored procedure that will be used by a web page for uploading images:

```
CREATE PROCEDURE WebUp
(
  @FileData varbinary(max)
)
AS

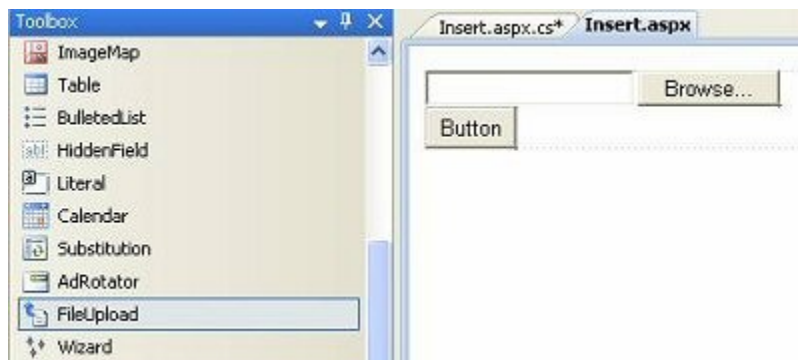
INSERT INTO BLOBFromWeb
  (BLOBData)
VALUES
  (@FileData);
```

The stored procedure will pass one variable, the image "@FileData", into SQL Server.

The rest of the examples in this series will use stored procedures rather than SQL statements. There are many benefits to using stored procedures, such as increased security, preventing SQL Injection attacks, portability, and performance.

Inserting into SQL from the Web

This example uses Microsoft Visual Studio 2008 to create a web form, which will ask the end user to browse to a file, and then upload the file into SQL Server. The application will also work in Visual Studio 2005. To begin, create a new web site and an aspx page using code behind. Drag a FileUpload and a Button control onto the form as shown below.



Switch to the source view of the page and change the default <form> tag to the following:

```
<form id="form1" runat="server" enctype="multipart/form-data">
```

The "enctype" specifies the how the form data is encoded.

Change back to the design view and double click the button to create an On Click event. This will bring up the code behind page as shown below:

```
public partial class Insert : System.Web.UI.Page
{
    protected void Page_Load(object sender, EventArgs e)
    {
    }

    protected void Button1_Click(object sender, EventArgs e)
    {
    }
}
```

At the top of the page, add using statements for the SQL:

```
using System.Data.Sql;  
using System.Data.SqlClient;
```

And also a statement for the file system:

```
using System.IO;
```

Add the following code into Button1_Click event:

```
string sConn = @"server=.; database=BLOBTest3; Integrated Security=True";  
  
SqlConnection objConn = new SqlConnection(sConn);  
objConn.Open();  
  
SqlCommand objCmd = new SqlCommand("WebUp", objConn);  
objCmd.CommandType = CommandType.StoredProcedure;  
  
SqlParameter paramFileData = objCmd.Parameters.Add("@FileData",  
SqlDbType.VarBinary);  
paramFileData.Direction = ParameterDirection.Input;  
  
byte[] bImage = new byte[FileUpload1.PostedFile.ContentLength];  
Stream objStream = FileUpload1.PostedFile.InputStream;  
objStream.Read(bImage, 0, FileUpload1.PostedFile.ContentLength);  
  
paramFileData.Value = bImage;  
  
objCmd.ExecuteNonQuery();  
objConn.Close();
```

The first line in the statement sets the connection to SQL Server. The phrase "Integrated Security" means Windows security will be used rather than a SQL Login ID. Next the connection to the database is opened. After the database is opened, the next two lines create a SQL Command object.

```
SqlCommand objCmd = new SqlCommand("WebUp", objConn);  
objCmd.CommandType = CommandType.StoredProcedure;
```

WebUp is the name of the stored procedure created earlier. It's tied to the open database connection, then the SQL Command is told WebUp is a stored procedure rather than a TSQL text statement.

The stored procedure requires a single parameter, the image, to be passed in. In the next line, a parameter is created. The name "paramFileData" can be anything, there is nothing special about it, but the Parameters.Add, "@FileData" must match the parameter name in the stored procedure.

```
SqlParameter paramFileData = objCmd.Parameters.Add("@FileData",  
SqlDbType.VarBinary);
```

Now, with the parameter created, the direction is specified:

```
paramFileData.Direction = ParameterDirection.Input;
```

Any data being passed into SQL is a command type of Input, data coming out would be ParameterDirection.Output.

Ideally, at this point, the FileUpload control could pass the image directly into the stored procedure parameter, but this doesn't work. Instead, a byte array is created with the size of the image:

```
byte[] bImage = new byte[FileUpload1.PostedFile.ContentLength];
```

Next a Stream is created pointing to the image content:

```
Stream objStream = FileUpload1.PostedFile.InputStream;
```

Lastly the Stream transfers the image into the byte array:

```
objStream.Read(bImage, 0, FileUpload1.PostedFile.ContentLength);
```

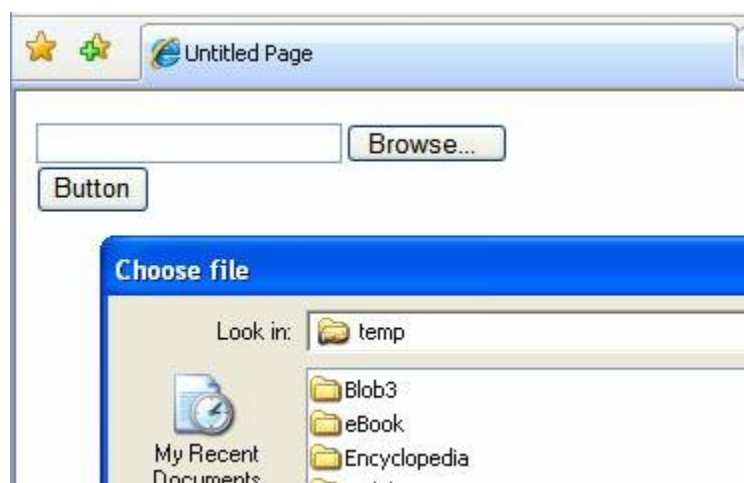
Now the image data can be passed into the SQL parameter and executed.

```
paramFileData.Value = bImage;
```

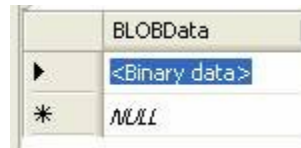
```
objCmd.ExecuteNonQuery();
```

Running the Application

View the web page inside a browser and click the browse button. A Windows file chooser will appear as shown below:



Once a file is selected and the final Button is clicked, the SQL Stored procedure will be executed. Viewing the results inside SQL Server will show there is data, but not what it looks like.



Because there isn't a BLOB viewer as part of SQL Server, we'll create a viewer similar to the one in the previous article, but stored procedure driven.

Viewer

Create a simple stored procure that will select the image data using the TSQL code below:

```
CREATE PROCEDURE BLOBViewer  
AS
```

```
SELECT BLOBData  
FROM BLOBFromWeb
```

Next create a aspx page with code behind. On the code behind page add using statements for SQL Server:

```
using System.Data.Sql;  
using System.Data.SqlClient;
```

In the Page_load section, use the following code to execute the stored procedure just created.

```
string sConn = @"server=.; database=BLOBTest3; Integrated Security=True";  
SqlConnection objConn = new SqlConnection(sConn);  
objConn.Open();
```

```
SqlCommand objCmd = new SqlCommand("BLOBViewer", objConn);  
objCmd.CommandType = CommandType.StoredProcedure;
```

```
SqlDataReader dr = objCmd.ExecuteReader();  
dr.Read();
```

```
Response.BinaryWrite((byte[])dr["BLOBData"]);
```

```
objConn.Close();
```

The web viewer code used is very similar to the previous article. The CommandType has been changed to Stored Procedure, and the SqlCommand now uses the name of the procedure rather than a sql string to execute.

Conclusion

SQL Server can be a handy container for image and BLOB data. Working with binary data is somewhat different from working with ASCII. The data saved inside SQL Server isn't visible like character data. In addition, special handling of the data, such as with a byte array, is

required before sending the data to SQL Server. Overall, however, the working with BLOBS and SQL Server is very worthwhile.